Full Stack Project

Web Engineer Assessment v2.3.1

1 Background

This document presents a mini-project assessment intended to evaluate the technical and problem-solving capabilities of candidates applying for Full-Stack Engineering roles at 101 Digital PTE LTD.

1.1 Approach

Prior to commencing the mini-project, candidates must thoroughly review and understand the following:

- The mini-project requirements set out in this document.
- The assessment criteria outlined below.
- The deliverables required for submission to 101 Digital.

1.2 Overview

Candidates are required to design and develop a full-stack web application named **SimpleInvoice**. The application must comprise a **ReactJS** frontend and a **NestJS** backend, supported by a database designed and implemented by the candidate.

No external third-party APIs are required for this project. Candidates are expected to develop and manage the complete application stack, including the frontend, backend, and database layers.

A mock dataset is provided in **Appendix A** to assist with database seeding and to provide guidance on the expected data structure and relationships.

2 Solution Requirements

2.1 The SimpleInvoice Overview

The SimpleInvoice web application shall provide the following core features:

1. **Authentication** – Secure user authentication and access control.
2. **Invoice Listing** – A view displaying all available invoices within the system.
3. **Invoice Details** – A detailed view presenting information associated with an individual invoice.
4. **Invoice Creation** – Functionality enabling users to create and save new invoices.

2.1.1 Authentication Requirements

Candidates must implement a secure authentication mechanism that satisfies the following requirements:

- Provide a Login screen containing **email address** and **password** input fields.
- Implement appropriate client-side and server-side validation for all authentication inputs.
- Upon successful authentication, issue a **JSON Web Token (JWT)** and securely store the token on the client side for use in subsequent API requests.
- Restrict access to all protected application routes and resources. Users who are not authenticated must be automatically redirected to the Login screen.

Note: The implementation of advanced password policies (e.g., complexity requirements, special character enforcement, password expiration, or multi-factor authentication) is not required as part of this assessment.

2.1.2 Invoice List (Home Screen)

- Set this as the default home screen on successful login
- Display a paginated list of invoices showing key fields: Invoice Number, Customer Name, Invoice Date, Due Date, Total Amount, Status
- Support the following list management features:
  - **Search** — by invoice number or customer name; case-insensitive, partial match supported
  - **Filter** — by invoice *status* (Draft, Pending, Paid, Overdue)
  - **Sort** — by invoiceDate, dueDate, or totalAmount (ascending / descending)
  - **Pagination** — server-side with a configurable page size.

Upon successful authentication, users shall be directed to the **Invoice List** screen, which serves as the default landing page of the application.

- The screen must display a paginated list of invoices, presenting the following key information for each record:
  - Invoice Number

- Customer Name
- Invoice Date
- Due Date
- Total Amount
- Invoice Status

The Invoice List must support the following data management capabilities:

- **Search**

  Users must be able to search invoices using either the **Invoice Number** or **Customer Name** fields. The search functionality must support:

  - Case-insensitive matching.
  - Partial text matching.

- **Filter**

  Users must be able to filter invoices by the following status values:

  - Draft
  - Pending
  - Paid
  - Overdue.

- **Sort**

  Users must be able to sort invoice records by any of the following fields:

  - Invoice Date
  - Due Date
  - Total Amount.

  Sorting must support both **ascending** and **descending** order.

- **Pagination**

  The invoice list must implement **server-side pagination**. The page size should be configurable and support efficient retrieval of invoice records from the backend.

2.1.3 Invoice Detail

Users must be able to view the details of an individual invoice by selecting an invoice from the Invoice List screen.

Upon selection, the application shall navigate the user to the **Invoice Details** view, which must display comprehensive information relating to the selected invoice, including:

- Invoice information
- Customer information
- Invoice line items
- Subtotal amount

- Tax amount
- Discount amount
- Total invoice amount
- Outstanding balance
- Invoice status

All invoice data presented in the detail view must accurately reflect the corresponding record stored in the system.

2.1.4 Create Invoice

- A screen/form to create a new invoice
- Each invoice contains **exactly one line item** for this assessment. The data model may be designed to support multiple items in the future, but only one item needs to be implemented.
- New invoices must be created with status Draft
- **Invoice number is user-provided and must be unique**
- Required fields and their validation rules:

Field	Validation
Customer name	Required, non-empty string
Customer email	Required, valid email format
Customer mobile	Optional
Customer address	Optional
Invoice number	Required, unique
Invoice date	Required, valid date
Due date	Required, must be on or after invoice date
Currency	Required (e.g. AUD, USD, GBP)
Item name	Required
Item quantity	Required, positive integer
Item rate	Required, positive number
Tax (%)	Non-negative number, defaults to 10%
Discount	Optional, non-negative number, defaults to 0

- On successful creation, show a success notification and redirect to the Invoice List
- **Total amount must be calculated by the backend**, not the frontend.

2.2 Frontend Implementation Requirements

The frontend application must be implemented in **ReactJS** using **TypeScript**.

The following requirements must be adhered to:

- The application must be fully responsive and support both **mobile** and **desktop** viewports.
- Candidates may use any preferred styling approach or library (e.g., CSS, SCSS, styled-components, or UI frameworks).
- The use of widely adopted open-source libraries is permitted and encouraged where appropriate.
- The codebase must be clean, maintainable, and adequately documented to support readability and future maintenance.
- Unit testing is mandatory. While no minimum test coverage threshold is enforced, candidates are expected to provide tests for critical user flows and key UI components.

2.3 Backend Implementation Requirements

The backend service must be implemented using **NestJS** with **TypeScript**, and must expose a **RESTful API**.

The application must be supported by a **relational database**, with **PostgreSQL** recommended as the preferred database system.

The backend implementation should ensure a clean, modular architecture and support reliable data persistence and retrieval for all application features.

2.3.1 API Endpoints

Method	Endpoint	Auth	Description
POST	`/auth/login`	✗	Authenticate user, return JWT
GET	`/auth/me`	✓	Return current authenticated user profile
GET	`/invoices`	✓	List invoices with search, filter, sort, pagination
GET	`/invoices/:id`	✓	Get invoice detail by ID
POST	`/invoices`	✓	Create a new invoice

Query parameters for GET /invoices:

Parameter	Type	Description
`page`	number	Page number, starting at 1
`pageSize`	number	Records per page
`sortBy`	string	invoiceDate, dueDate, totalAmount
`ordering`	string	ASC or DESC
`status`	string	Draft, Pending, Paid, Overdue

Parameter	Type	Description
`keyword`	string	Partial, case-insensitive search on invoice number or customer name
`fromDate`	string	Filter invoices on/after this date (YYYY-MM-DD)
`toDate`	string	Filter invoices on/before this date (YYYY-MM-DD)

Expected response shape for GET /invoices:

```
{
  "data": [...],
  "paging": {
    "page": 1,
    "pageSize": 10,
    "total": 100
  }
}
```

2.3.2 Business Logic

Total amount calculation must be done on the *server-side*:

```
subTotal      = quantity × rate
taxAmount     = subTotal × (tax% / 100)
totalAmount   = subTotal + taxAmount - discount
balanceAmount = totalAmount - totalPaid
```

- Invoice numbers must be unique — enforced at the database level
- Due date must be on or after invoice date — validated server-side
- New invoices are always created with status Draft

Overdue status behaviour:

Overdue is a **derived status** — it is not persisted in the database. The backend computes it at read time:

```
if status != "Paid" AND dueDate < today  →  return status as "Overdue"
otherwise                                →  return the persisted status
```

The database stores only Draft, Pending, or Paid. Overdue is never written to the database.

2.3.3 Authentication & Authorization

The backend must implement authentication and authorization using **JSON Web Tokens (JWT)** to support stateless session management.

The following requirements apply:

- Authentication must be based on **JWT access tokens**.
- All /invoices API endpoints must be secured using a **JWT authentication guard**.
- Token expiration time must be configurable via an environment variable, with a default value of **3600 seconds** if not explicitly specified.

- At least one default user account must be seeded into the database for reviewer access. The corresponding credentials must be clearly documented in the project **README file**.

2.3.4 Data Seeding

- The backend must include a database seed script responsible for populating the system with sample invoice data.
- The following requirements apply:
- A seed script must be provided to populate the database with initial dataset entries.
- The mock dataset provided in **Appendix A** must be used as the foundation for seed data structure and relationships.
- Additional invoice records (approximately **20–50 entries**) must be generated to ensure sufficient data variability for testing.
- Generated records must include a diverse range of:
- Invoice statuses
- Invoice dates
- Due dates
- Total amounts
- This is to ensure that key features such as search, filtering, sorting, and pagination can be effectively demonstrated and evaluated.
- The seed script must be runnable with a single command:

```
npm run seed
```

2.3.5 Input Validation

- Use `class-validator` + `class-transformer` (NestJS ValidationPipe) on all endpoints
- Return clear, structured validation error responses:

```
{
  "statusCode": 400,
  "message": ["dueDate must be on or after invoiceDate"],
  "error": "Bad Request"
}
```

2.3.6 Error Handling

- Implement a global exception filter returning consistent error responses:

```
{
  "statusCode": 404,
  "message": "Invoice not found",
  "error": "Not Found"
}
```

2.3.7 Testing Requirements

Unit testing is mandatory for the backend implementation. While no minimum test coverage threshold is enforced, candidates are expected to ensure adequate coverage of core functionality.

The following requirements must be satisfied:

- Unit tests must be implemented for the backend application.
- Critical business logic must be covered by tests, including:
  - Invoice total calculations
  - Overdue status derivation
  - Due date validation logic
  - Enforcement of unique invoice numbers
- At least one integration or end-to-end test must be implemented, covering a complete key workflow (for example: creating an invoice and verifying its appearance in the invoice list).

2.3.8 API Documentation

The backend API must be documented using **Swagger/OpenAPI**, generated via the `@nestjs/swagger` package.

The following requirements apply:

- API documentation must be automatically generated using `@nestjs/swagger`.
- The Swagger UI must be available at `/api/docs` when the application is running.
- All API endpoints must be fully documented, including:
  - Request payloads
  - Query parameters
  - Response schemas and status codes.

2.4 Architecture Requirements

2.4.1 Project Structure

Organize your project as a *monorepo* or two *separate* repos. Either is acceptable — document your choice in the README.

Suggested monorepo structure:

```
simple-invoice/
├── frontend/               # ReactJS app
│   ├── src/
│   └── ...
├── backend/                # NestJS API
│   ├── src/
│   │   ├── auth/
│   │   ├── invoices/
│   │   ├── database/
│   │   │   └── seed/
│   │   └── common/
│   └── ...
├── docker-compose.yml
└── README.md
```

2.4.2 Containerization

- Provide a docker-compose.yml that starts the frontend, backend, and database with a single command:

```
docker compose up        # modern Docker
# or
docker-compose up        # legacy Docker Compose
```

- Each service must have its own Dockerfile
- Document all exposed ports in the README

2.4.3 Environment Configuration

The application must support environment-based configuration using .env files for all environment-specific settings.

The following requirements apply:

- All environment-specific configuration values (including database connection strings, JWT secrets, and application ports) must be managed via .env files.
- A .env.example file must be provided, containing all required configuration keys without any real or sensitive values.
- All secrets, credentials, and configuration parameters must be sourced exclusively from environment variables.
- Hardcoding of sensitive information within the codebase is strictly prohibited.

3 Data Model Reference

Use the below as a guide when designing the database schema.

3.1 Invoice

Field	Type	Notes
`invoiceId`	UUID	Primary key, generated by backend
`invoiceNumber`	string	Unique, user-provided
`invoiceReference`	string	Optional external reference
`invoiceDate`	date	Required
`dueDate`	date	Required, must be ≥ invoiceDate
`currency`	string	ISO 4217 code, e.g. AUD
`currencySymbol`	string	Display symbol, e.g. AU$
`description`	string	Optional
`status`	enum	Persisted: Draft, Pending, Paid. Overdue is derived at read time.
`invoiceSubTotal`	decimal	quantity × rate
`totalTax`	decimal	Calculated server-side
`totalDiscount`	decimal	Calculated server-side
`totalAmount`	decimal	Final payable amount
`totalPaid`	decimal	Amount paid so far
`balanceAmount`	decimal	totalAmount - totalPaid
`createdAt`	datetime	Auto-set on creation
`createdBy`	UUID	FK to User

3.2 Customer (embedded in Invoice)

Field	Type	Notes
`fullname`	string	Required
`email`	string	Required, valid email
`mobileNumber`	string	Optional
`address`	string	Optional

> *Customer may be stored as embedded fields on the Invoice table, or as a separate customers table — either approach is acceptable. Document your choice.*

3.3 Invoice Item

Field	Type	Notes
`id`	UUID	Primary key
`invoiceId`	UUID	FK to Invoice
`name`	string	Required
`quantity`	integer	Required, positive
`rate`	decimal	Required, positive

3.4 User

Field	Type	Notes
`id`	UUID	Primary key
`email`	string	Unique, used as login identifier
`passwordHash`	string	Bcrypt hashed
`fullname`	string	Display name
`createdAt`	datetime	Auto-set

4 Assessment Criteria & Deliverables

4.1 Assessment Criteria

Criteria	Description
Packaging	Easy to clone, run, and review — minimal setup steps
Working and complete app	All four features functional as per specification
API Design	Clean REST API with consistent response shapes and appropriate HTTP status codes
Database Design	Sensible schema, appropriate constraints and indexes
Business Logic	Correct server-side calculation, Overdue derivation, validation rules enforced
Authentication	Secure JWT implementation, all protected routes guarded
Code Quality	Clean, readable, well-organised, follows industry best practices
Testing	Business logic and key API flows covered by tests
Documentation	Clear README, working Swagger docs, documented assumptions
Docker Setup	Single command brings up the full stack from zero
Value Add	Additional features or improvements beyond the stated requirements

4.2 Deliverables

Candidates must ensure that all submissions are completed and sent no later than the specified due date and time.

All submission materials, including links to code repositories, must be sent to the designated submission contact provided below.

- ThanhNguyenBa@101digital.io
- rajiv@101digital.io

The submission must include the following details to ensure proper identification and tracking:

- Git repository ID of the submitted project
- Candidate's email address used for the submission

Note: The submission must include the following components:

- Source code, provided via a **GitHub** or **GitLab** repository (preferred), or as a compressed ZIP archive.
- A README.md file located at the root of the project, which must include the following information:
  - Project overview and architecture description
  - Instructions for running the application locally, both with and without Docker
  - Default login credentials for reviewer access
  - Instructions for executing the database seed script
  - Any assumptions or design decisions made during implementation
  - A list of known limitations or incomplete features within the implementation

Appendix A — Mock Dataset

Use the following as the basis for your seed script. The quantity field is an integer; the unit of measure is not part of the data model.


```
{
  "data": [
    {
      "createdAt": "2026-06-03T12:03:26.995",
      "createdBy": "ad1e0902-1928-4345-b513-60c86c94fc91",
      "currency": "AUD",
      "currencySymbol": "AU$",
      "customer": {
        "fullname": "Paul",
        "address": "Singapore",
        "email": "paul@101digital.io",
        "mobileNumber": "947717364111"
      },
      "description": "Invoice is issued to Kanglee",
      "dueDate": "2026-07-03",
      "invoiceDate": "2026-06-03",
      "invoiceId": "099ca7da-a290-40fa-93b9-1c43ae7bb887",
      "invoiceNumber": "IV1780488206995",
      "invoiceSubTotal": 2000.00,
      "totalDiscount": 20.00,
      "totalTax": 200.00,
      "totalAmount": 2180.00,
      "totalPaid": 1451.34,
      "balanceAmount": 728.66,
      "items": [
        {
          "id": "b1c2d3e4-0000-0000-0000-000000000001",
          "name": "Honda RC150",
          "quantity": 2,
          "rate": 1000
        }
      ],
      "invoiceReference": "#5721662",
      "status": "Overdue",
      "type": "INVOICE",
      "invoiceGrossTotal": 2000.00
    }
  ],
  "paging": {
    "pageNumber": 1,
    "pageSize": 10,
    "totalRecords": 94980
```

```
    }
}
```

> *Seed guidance: Generate 20–50 additional records with a mix of statuses (Draft, Pending, Paid), varied invoice dates, due dates, amounts, and customer names so that all list management features — search, filter, sort, and pagination — are meaningful to test. Do not seed Overdue as a stored status; it will be derived automatically.*